AI TECHNICAL INTERVIEWER (RAG v2.1)

Deep Technical Architecture, Mathematical Foundations & Pipeline Mechanics

Platform: Autonomous Agentic RAG Platform | **Vector DB:** FAISS FlatIP | **Embeddings:** all-MiniLM-L6-v2 (d=384) | **LLM Judge:** Gemini Flash Lite

1. Executive Overview & Problem Formulation

Standard Large Language Models (LLMs) evaluated in technical interview preparation face two fundamental weaknesses: **(1) Parametric Hallucination:** LLMs generate plausible but factually incorrect code, complexity bounds, and algorithmic steps without grounding; and **(2) Static Knowledge Decay:** LLMs cannot dynamically incorporate cutting-edge coding repositories or real-time online technical updates. This project implements an **Offline-First, Dual-Source Retrieval-Augmented Generation (RAG) Platform** designed to dynamically discover, index, retrieve, formulate, and evaluate technical interview problems strictly grounded in verifiable source documents.

2. End-to-End Pipeline Execution Stages

Stage	Component / Layer	Underlying Algorithm / Technique	Output / Artifact
1. Ingestion	Hybrid Dual-Tier Store	Tier-1 Local JSON DB + Tier-2 GitHub API Crawler & GFG Parser	Raw technical Markdown/code blocks
2. Chunking	Sliding Window Chunker	1000-character chunk size with 200-character overlapping sliding window	Segmented semantic chunks with metadata
3. Embedding	SentenceTransformer	all-MiniLM-L6-v2 mapping text to $\blacksquare^{384}$ + L_2 unit normalization	Normalized dense vectors $\|\|v\|\|_2 = 1.0$
4. Indexing	FAISS Vector Index	faiss.IndexFlatIP (Inner Product on L_2 normalized vectors)	Fast exact cosine similarity search space
5. Retrieval	Threshold Routing Engine	Top-K (K=5) Nearest Neighbor search; similarity threshold ≥ 0.52 cutoff	Top-5 relevant context blocks (or online trigger)
6. Synthesis	Context-Bounded Prompter	Strict ground truth injection + dynamic caching to dynamic_cache.json	Context-grounded interview challenge
7. Evaluation	Single-Pass LLM Judge	4-part weighted 10-pt rubric + co-generated adaptive follow-up probe	Assessment report + instant follow-up (0s)

3. Mathematical & Algorithmic Foundations

A. Vector Embedding & L_2 Normalization

Text chunks are projected into a continuous vector space using the sentence-transformers/all-MiniLM-L6-v2 model. Given a text string *T*, the encoder produces a 384-dimensional dense vector $\mathbf{e} = f_\theta(T) \in \blacksquare^{384}$. To enable ultra-fast cosine similarity calculations, every vector is explicitly normalized to unit length using Euclidean L_2 normalization:

$$\text{L}_2\ \textbf{Normalization Formula:} \quad \mathbf{u} = \mathbf{e} \,/\, ||\mathbf{e}||_2 = \mathbf{e} \,/\, \sqrt{\left(\sum_{i=1..384} e_i^2\right)} \quad \Rightarrow \quad ||\mathbf{u}||_2 = 1.0$$

B. FAISS Cosine Similarity via Inner Product Equivalence

Standard Cosine Similarity between a query vector **q** and document vector **d** is defined as:

Cosine Similarity(q, d) = $(\mathbf{q} \cdot \mathbf{d}) \,/\, (||\mathbf{q}||_2 \times ||\mathbf{d}||_2)$

Because all document chunks and incoming queries are pre-normalized to unit Euclidean length ($||\mathbf{q}||_2 = 1$ and $||\mathbf{d}||_2 = 1$), the denominator reduces strictly to 1.0. Thus, **Cosine Similarity is identical to the Inner Product**:

$$\textbf{Inner Product Equivalence:} \quad \text{CosineSim}(\mathbf{q}, \mathbf{d}) = \mathbf{q} \cdot \mathbf{d} = \sum_{i=1..384} (q_i \times d_i)$$

By utilizing faiss.IndexFlatIP, the system computes exact cosine similarities across thousands of chunks via single BLAS matrix multiplications, achieving sub-millisecond retrieval with zero quantization loss.

4. Chunking Strategy & Threshold Routing Logic

Sliding Window with Boundary Overlap: Documents are divided into chunks of **1,000 characters** with a **200-character overlap**. The overlap guarantees that multi-line algorithmic invariants (e.g., recursive base cases, loop termination invariants, and lock acquisition routines) are not sliced in half across arbitrary character splits.

Similarity Threshold Routing (Threshold = 0.52):

• **Cosine Similarity ≥ 0.52 (Cache Hit):** The topic exists in the local store. The top 5 chunks are retrieved instantly with **0ms network latency** and **zero API cost**.

• **Cosine Similarity < 0.52 (Cache Miss):** Triggers Tier-2 Online Discovery. The system ranks top GitHub interview repositories (scored by stars, forks, and keyword density) and parses GeeksforGeeks technical articles, then automatically saves the new chunks into dynamic_cache.json for instant future retrieval.

5. Candidate Evaluation Parameters & Weighted Rubric

Candidate responses are judged strictly against the retrieved technical ground truth. The system evaluates answers across **4 explicit weighted parameters** totaling **10.0 points**:

Parameter Dimension	Weight	Max Points	Evaluation Criteria & Focus
1. Conceptual Correctness & Invariants	40%	4.0 / 10	Selection of appropriate data structures (e.g., Queue vs Stack for BFS, Hash Map + DLL for LRU Cache); preserving core algorithmic invariants without anti-patterns.
2. Completeness, Edge Cases & Constraints	25%	2.5 / 10	Boundary conditions (left ≤ right), cycle handling via visited set, null/empty collections, duplicate values, memory leak prevention, recursion base cases.
3. Complexity Analysis (Time & Space)	20%	2.0 / 10	Accurate Big-O asymptotic runtime (O(V+E), O(log n)); correct distinction between auxiliary space (call stack frames) vs total space.
4. Technical Precision & Communication	15%	1.5 / 10	Correct engineering terminology, clear non-contradictory logic, sound justification for design tradeoffs.

Score-to-Verdict & Adaptive Follow-up Mapping:

• **Score ≥ 7.5 (Verdict: Correct):** Meets core invariants and complexity bounds. Triggers **advanced deep-dives** (e.g., scale optimization, 0-1 BFS, concurrency).

• **Score 5.0 – 7.4 (Verdict: Partially Correct):** General approach sound, but missing an invariant or boundary check. Triggers a **targeted probe**.

• **Score < 5.0 (Verdict: Incorrect):** Fundamental technical misconception detected (e.g. Stack used for BFS). Triggers **foundational remedial correction**.

6. Latency Optimizations & Performance Benchmarks

To achieve a fluid, real-time interview experience, the system implements three core architectural latency optimizations:

1. Single-Pass Co-generation: Rather than executing sequential LLM calls (one for grading, followed by another for follow-up generation), the evaluator synthesizes the grading report and the adaptive follow-up probe in a single API call, reducing the first follow-up wait time to **0.0s**.

2. Speculative Multi-Turn Prefetching: Follow-up answers are evaluated while speculatively pre-generating the subsequent challenge probe in a FollowUpFeedback container, eliminating wait time on multi-turn iterations.

3. High-Speed Flash Model Tiering: Prioritizes gemini-flash-lite-latest with instant 503 failover, bypassing heavy server queue delays.

Pipeline Operation	Previous Latency	Optimized Latency	Performance Gain
LLM API Round-Trip	22.0s – 95.3s (gemini-3.6-flash queue)	**1.0s – 1.6s** (flash-lite)	**~50x Speedup**
Initial Answer Evaluation	22.0s – 95.0s	**~2.7s**	**~35x Speedup**
First Follow-up Probe Display	3.0s – 5.0s (sequential call)	**0.0s (Instant Co-generated)**	**Latency Eliminated**
Follow-up Answer Review	15.0s – 30.0s	**~1.8s**	**~15x Speedup**
Multi-turn Next Probe Display	3.0s – 5.0s (sequential call)	**0.0s (Instant Prefetched)**	**Latency Eliminated**